

Lecture: Huffman Encoding and Data Compression

Date: August 22, 2026
Instructor: Aniket Basu Roy
Scribe By: Naisargi(2026H1030044G), Prabal(2026H1030027G)

Overview. In this lecture, we study the Data Compression Problem using Huffman Encoding. We present the Huffman algorithm, illustrate its working with an example, and analyze its time and space complexity.

1 The Data Compression Problem

Motivation. Data compression aims to represent information using fewer bits while still allowing the original data to be reconstructed. Since some symbols occur more frequently than others, assigning shorter codes to frequent symbols and longer codes to less frequent symbols can reduce the total number of bits required.

Definition 1 (Instance). An instance consists of a set S of symbols and a frequency $f(x)$ for each $x \in S$, where $f(x)$ denotes the number of times symbol x occurs in the input.

Definition 2 (Encoding). An *encoding* is a function

$$\gamma : S \rightarrow \{0, 1\}^*,$$

where $\gamma(x)$ is the binary representation assigned to each symbol $x \in S$.

Definition 3 (Prefix code). An encoding γ is a *prefix code* if

$$\forall x, y \in S, \quad x \neq y \implies \gamma(x) \text{ is not a prefix of } \gamma(y).$$

Thus, no codeword is the prefix of another, allowing the encoded sequence to be decoded uniquely.

Definition 4 (Huffman encoding). For each $x \in S$, let $|\gamma(x)|$ denote the number of bits used to represent x . The goal of Huffman encoding is to find a prefix encoding γ that minimizes the total encoded length

$$\min_{\gamma} \sum_{x \in S} f(x) |\gamma(x)|.$$

2 Worked Example

Given frequencies:

$$f_a = 0.32, \quad f_b = 0.25, \quad f_c = 0.20, \quad f_d = 0.18, \quad f_e = 0.05$$

The two least frequent symbols e and d are first combined to form a node of frequency 0.23. This node is then combined with c to form a node of frequency 0.43. Similarly, a and b are combined to form a node of frequency 0.57. Finally, the nodes of frequencies 0.43 and 0.57 are combined to form the root.

Assigning 0 to a left edge and 1 to a right edge gives the following Huffman codes:

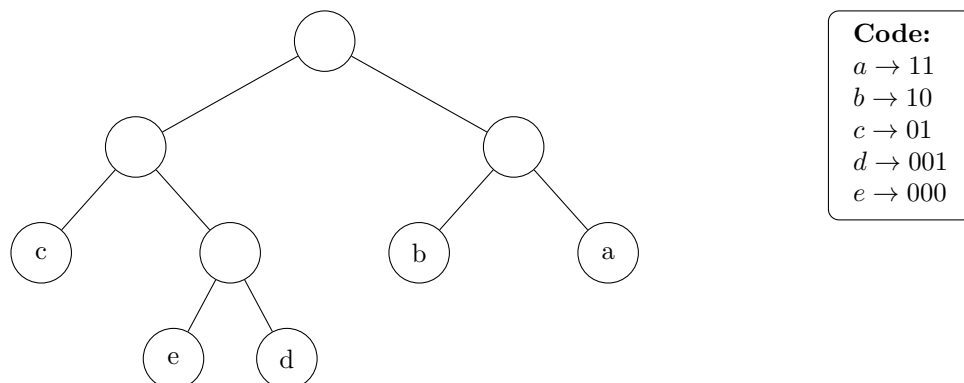


Figure 1: Final Huffman Tree

3 Huffman Encoding Algorithm

Algorithm 1: Huffman Encoding

Input: A set S of symbols with frequency $f(x)$ for each $x \in S$ **Output:** A Huffman tree T

```
1 Initialize a min-heap  $Q$  containing one node for each  $x \in S$ , with key  $f(x)$ 
2 while  $|Q| > 1$  do
3    $x \leftarrow \text{Extract-Min}(Q)$ 
4    $y \leftarrow \text{Extract-Min}(Q)$ 
5   Create a new node  $z$ 
6    $z.\text{left} \leftarrow x$ 
7    $z.\text{right} \leftarrow y$ 
8    $f(z) \leftarrow f(x) + f(y)$ 
9   Insert( $Q, z$ )
10  $T \leftarrow \text{Extract-Min}(Q)$ 
11 return  $T$ 
```

Algorithm 1 starts by inserting all symbols into a min-heap using their frequencies as keys. At each step, the two nodes with the smallest frequencies are removed and combined into a new node whose frequency is their sum. The new node is then inserted back into the heap. This process continues until only one node remains, which becomes the root of the Huffman tree.

4 Efficiency Analysis

Let $n = |S|$ be the number of distinct symbols. The initial construction of the min-heap takes $O(n)$ time. The loop performs $n - 1$ iterations, and each iteration performs two Extract-Min operations and one insertion, each taking $O(\log n)$ time. Therefore, the total running time is

$$O(n) + O(n \log n) = O(n \log n).$$

The algorithm uses $O(n)$ space for the min-heap and the Huffman tree.